

SymPy Bug Report

1 Bug 38: Spurious Imaginary Part in a Divergent Real Integral

1.1 Minimal Reproducer

```
from sympy import Abs, Rational, integrate, symbols

x = symbols("x", real=True)
val = integrate(1/Abs(x), (x, -1, 0))
print("integral =", val)
print("truncated eps=1/10:",
      integrate(1/Abs(x), (x, -1, -Rational(1, 10))))
print("truncated eps=1/100:",
      integrate(1/Abs(x), (x, -1, -Rational(1, 100))))
```

1.2 Observed Behavior

```
integral = oo + I*pi
truncated eps=1/10: log(10)
truncated eps=1/100: log(100)
```

SymPy reports the improper integral as $\infty + i\pi$. The finite truncations are real and positive, and the integrand is real and positive on the interval of integration except at the singular endpoint. The imaginary term is therefore not a harmless alternate representation of the real improper integral.

1.3 Expected Behavior

The mathematically correct result is positive real infinity:

$$\int_{-1}^0 \frac{1}{|x|} dx = \infty.$$

The result should not contain any finite imaginary component.

1.4 Mathematical Explanation

For $x \in [-1, 0)$, $|x| = -x$, so

$$\frac{1}{|x|} = -\frac{1}{x}.$$

The corresponding truncated improper integral is

$$\int_{-1}^{-\varepsilon} \frac{1}{|x|} dx = \int_{-1}^{-\varepsilon} -\frac{1}{x} dx = -\log(\varepsilon), \quad \varepsilon > 0.$$

As $\varepsilon \rightarrow 0^+$, this tends to $+\infty$. More generally,

$$\int_{-a}^{-\varepsilon} \frac{1}{|x|} dx = \log\left(\frac{a}{\varepsilon}\right) \rightarrow +\infty \quad (a > 0).$$

The integral is over a real-valued, nonnegative function on the real line. No branch choice for the complex logarithm should contribute a residual $i\pi$ to this real improper integral.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant public integration path as `Integral.doit` in `sympy/integrals/integrals.py`. Because the integrand contains `Abs`, the integrator rewrites $1/\text{Abs}(x)$ as a real `Piecewise` expression, equivalent to

$$\text{Piecewise}((1/x, x \geq 0), (-1/x, \text{True})).$$

The `Piecewise` antiderivative path then produces an antiderivative equivalent to

$$\text{Piecewise}((- \log(x), x \leq 0), (\log(x), \text{True})).$$

Definite interval evaluation proceeds through `Integral.doit` in `sympy/integrals/integrals.py`, which calls `_eval_interval` on the antiderivative. The diagnosis locates the responsible endpoint behavior in `_eval_interval` in `sympy/core/expr.py`, used through `Piecewise._eval_interval` in `sympy/functions/elementary/piecewise.py`.

The problematic rule is that endpoint evaluation first substitutes the endpoint into the antiderivative and only falls back to a one-sided limit if that substituted value is infinite or not a number:

```
C = self.subs(x, c)
```

At the finite endpoint $x = -1$, substituting into the negative-branch antiderivative $-\log(x)$ gives $-i\pi$, using the principal complex logarithm. Since this is finite, `_eval_interval` keeps it. At the singular endpoint $x \rightarrow 0^-$, the one-sided limit is ∞ . The interval evaluation therefore forms

$$\infty - (-i\pi) = \infty + i\pi.$$

This source-level behavior evaluates a real-branch antiderivative using the principal complex value of $\log(x)$ at a negative real endpoint. On the negative real interval, an antiderivative for $-1/x$ should behave like $-\log(-x)$, or equivalently use a real branch value up to a real constant. A plausible fix direction supported by the diagnosis is to evaluate `Piecewise` antiderivative endpoints by one-sided limits within the active real branch, or to represent this negative-branch antiderivative in a real-valued form such as $-\log(-x)$ or $-\log(|x|)$.

1.6 Additional Failing Instances

The same failure occurs for the family

$$\int_{-a}^0 \frac{1}{|x|} dx \quad (a \in \{1, 2, 3, 4, 5, 6\}).$$

For every positive a , the integrand is real and nonnegative on $[-a, 0)$, and the truncated integral $\int_{-a}^{-\varepsilon} 1/|x| dx = \log(a/\varepsilon)$ diverges to $+\infty$. Thus the expected result is ∞ with no imaginary part throughout the family.

Input / Expression	SymPy behavior	Expected behavior
$\int_{-1}^0 1/ x dx$	$\infty + i\pi$	∞
$\int_{-2}^0 1/ x dx$	not equal to ∞ ; includes a spurious imaginary branch contribution	∞
$\int_{-3}^0 1/ x dx$	not equal to ∞ ; includes a spurious imaginary branch contribution	∞
$\int_{-4}^0 1/ x dx$	not equal to ∞ ; includes a spurious imaginary branch contribution	∞
$\int_{-5}^0 1/ x dx$	not equal to ∞ ; includes a spurious imaginary branch contribution	∞
$\int_{-6}^0 1/ x dx$	not equal to ∞ ; includes a spurious imaginary branch contribution	∞

1.7 Related Issues Assessment

The bug-finding harness performed a deduplication check. The closest related family is improper real integration evaluated through logarithmic antiderivatives across a branch cut, and this failure mode is already publicly reported in the open issue [SymPy #23337](#) (“Spurious $I\pi$ in improper definite integral”). That issue documents the same root cause: for `integrate(1/(x-3), (x, 1, 3))`, the logarithmic antiderivative $\log(x - 3)$ evaluates to $\log(2) + i\pi$ at the finite endpoint via the principal complex log, and this $i\pi$ branch constant fails to cancel during definite interval evaluation — exactly the `C = self.subs(x, c)` mechanism diagnosed here. The deduplication verdict is therefore a specific new instance of an already-reported defect: the concrete `integrate(1/Abs(x), (x, -1, 0))` reproducer returning $\infty + i\pi$ does not appear verbatim in #23337, but it shares the same root cause and should be cross-referenced against #23337 rather than filed as fully novel.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Abs, integrate, oo, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_improper_integral_abs_negative_side_has_no_imaginary_part(a):
    x = symbols("x", real=True)
    assert integrate(1/Abs(x), (x, -a, 0)) == oo
```